



PyQuest

Information- & Aufgabenblatt

Kapitel 10: for-Schleifen

Laufe elegant über Zahlenreihen und Listen – mit `for`, `range()` und `break/continue`.

for-Schleifen und range()

Eine **for-Schleife** wiederholt Code **für jedes Element einer Reihe** – zum Beispiel für jedes Element einer **Liste** (die kennst du schon aus dem letzten Kapitel) oder für jede Zahl einer Zahlenreihe.

Eine Zahlenreihe erzeugst du mit **range()**: **range(n)** liefert die Zahlen von 0 bis $n - 1$. Der große Vorteil gegenüber `while`: Du musst **nichts selbst hochzählen** – die Laufvariable bekommt bei jedem Durchlauf automatisch den nächsten Wert.

In der nächsten Lektion lässt du `for` dann direkt über deine **Listen** laufen.

`range(5)` zählt von 0 bis 4:

Beispiel

```
for i in range(5):  
    print(i)
```

`range()` kann auch mit **Start und Ende** aufgerufen werden: **range(1, 6)** zählt von 1 bis 5.

Merke: Das Ende ist immer **ausgeschlossen** – die 6 kommt also nicht mehr dran. Willst du bis 10 zählen, brauchst du `range(1, 11)`.

Aufgabe 1: Wie viele Zahlen gibt `for i in range(1, 6)` aus?

- ☐ a) 4
- ☐ b) 5
- ☐ c) 6



Mit einem **dritten Wert** legst du die **Schrittweite** fest: `range(start, ende, schritt)`.

- `range(0, 10, 2)` → 0, 2, 4, 6, 8 (in Zweierschritten)

- `range(10, 0, -1)` → 10, 9, 8, ... , 1 (**rückwärts** mit negativem Schritt)

Achtung: `range(1, 11, -1)` erzeugt **gar keine** Zahlen – von 1 aus kommt man mit -1 nie bei 11 an.

Aufgabe 2: Wie oft wird die Schleife `for i in range(0, 5, 2):` ausgeführt?

- ☐ a) 2-mal
- ☐ b) 3-mal
- ☐ c) 5-mal

Aufgabe 3: Gib mit einer `for`-Schleife die Zahlen von 1 bis 10 aus (jede in einer eigenen Zeile).

Dein Code:

Auch mit `for` kannst du bequem **aufsummieren**: Ein Summierer vor der Schleife, in jedem Durchlauf `+=`. Weil `for` das Hochzählen selbst übernimmt, ist der Code oft kürzer als mit `while`.

Aufgabe 4: Berechne mit einer `for`-Schleife die Summe aller Zahlen von 1 bis 100 und gib nur das Ergebnis aus.

Erwartet: **5050**

Dein Code:



for-Schleife über Listen

Erinnerst du dich, wie mühsam es im Listen-Kapitel war, eine Liste mit `while`, einer Index-Variable und `len()` durchzulaufen? Jetzt kommt der elegante Weg.

Die `for`-Schleife und **Listen** sind ein perfektes Team. Du kannst mit `for` **direkt über die Elemente einer Liste** laufen – ganz ohne `range()` und ohne Index:

```
for element in liste:  
    # element ist bei jedem Durchlauf das nächste Listenelement
```

Bei jedem Durchlauf steckt im Laufwort (hier `element`) automatisch das **nächste Element** der Liste.

Über eine Liste von Namen laufen und jeden begrüßen:

Beispiel

```
namen = ["Anna", "Ben", "Carla"]  
for name in namen:  
    print("Hallo", name)
```

Aufgabe 5: Was steckt bei `for zahl in [2, 4, 6]: nacheinander in zahl`?

- ☐ a) Die Indizes 0, 1, 2
- ☐ b) Die Werte 2, 4, 6
- ☐ c) Die ganze Liste [2, 4, 6]

Aufgabe 6: In der Liste `farben` stehen drei Farben. Gib mit einer `for`-Schleife jede Farbe in einer eigenen Zeile aus.

```
farben = ["rot", "gelb", "grün"]
```

Dein Code:



Sehr oft willst du die Elemente einer Liste **auswerten** – zum Beispiel **aufsummieren**. Dazu kombinierst du die Liste mit einem **Summierer** (`summe = 0`), wie du ihn schon von den while-Schleifen kennst.

Aufgabe 7: In der Liste `zahlen` stehen sechs Zahlen. Berechne mit einer for-Schleife ihre Summe und gib nur das Ergebnis aus.

Erwartet: **108**

```
zahlen = [4, 8, 15, 16, 23, 42]
```

Dein Code:

Du kannst dir während des Durchlaufs auch etwas **merken** – zum Beispiel die bisher größte Zahl. Der Trick: Starte mit dem **ersten** Element (`zahlen[0]`) als bisherigem Maximum und prüfe für jedes weitere Element, ob es **größer** ist. Wenn ja, merkst du es dir.

Aufgabe 8: In der Liste `zahlen` stehen fünf Zahlen. Finde mit einer for-Schleife die größte Zahl und gib sie aus (ohne die fertige Funktion `max()` zu benutzen).

Erwartet: **19**

```
zahlen = [7, 2, 19, 5, 11]
```

Dein Code:



for-Schleife anwenden

In einer Schleife kannst du für **jede Zahl** eine Prüfung machen – dazu setzt du ein **if** in die Schleife. Besonders praktisch ist der **Modulo-Operator %**, der den **Rest** einer Division liefert:

- `zahl % 2 == 0` → die Zahl ist **gerade**
- `zahl % i == 0` → die Zahl ist **ohne Rest teilbar** durch `i` (also `i` ist ein **Teiler**)

Alle Teiler von 12 finden – für jede Zahl von 1 bis 12 wird geprüft, ob 12 ohne Rest teilbar ist:

Beispiel

```
zahl = 12
for i in range(1, zahl + 1):
    if zahl % i == 0:
        print(i)
```

Aufgabe 9: Frage mit Zahl: eine positive ganze Zahl ab. Gib alle Teiler dieser Zahl aus (jeden in einer eigenen Zeile).

Beispiel: Eingabe **12** → 1, 2, 3, 4, 6, 12 (untereinander).

Dein Code:



Du kannst in der Schleife auch **je nach Prüfung etwas anderes** ausgeben. Ein Klassiker ist „**Fizz**“: Bei jeder Zahl, die durch 3 teilbar ist, wird statt der Zahl das Wort Fizz ausgegeben – sonst die Zahl selbst.

Dazu kombinierst du for, if (mit %) und else.

Aufgabe 10: Gib die Zahlen von 1 bis 15 aus (jede in einer eigenen Zeile). Ist eine Zahl durch 3 teilbar, gib stattdessen Fizz aus.

Erwartet: 1, 2, Fizz, 4, 5, Fizz, 7, 8, Fizz, 10, 11, Fizz, 13, 14, Fizz

Dein Code:

break und continue

Mit der **Schleifensteuerung** greifst du in den Ablauf einer Schleife ein:

- **break** beendet die Schleife **sofort komplett** – das Programm macht danach hinter der Schleife weiter.
- **continue** überspringt nur den **Rest des aktuellen Durchlaufs** und springt direkt zum nächsten.

break: Sobald i die 13 erreicht, wird die Schleife verlassen – 13 und 14 werden nicht mehr ausgegeben.

Beispiel

```
for i in range(10, 15):  
    if i == 13:  
        break  
    print(i)
```

continue: Die 13 wird übersprungen, danach läuft die Schleife normal weiter.

Beispiel

```
for i in range(10, 15):  
    if i == 13:  
        continue  
    print(i)
```



Wichtig: break und continue benutzt man fast immer **zusammen mit einem if**. Ein break oder continue ganz ohne Bedingung würde ja schon beim **allerersten** Durchlauf wirken – das ergäbe selten Sinn. Die if-Bedingung entscheidet, **wann** die Steuerung greift.

Aufgabe 11: Was ist der Unterschied zwischen break und continue?

- ☐ a) Beide beenden die Schleife komplett
- ☐ b) break beendet die Schleife komplett, continue überspringt nur den aktuellen Durchlauf
- ☐ c) break überspringt nur einen Durchlauf, continue beendet die Schleife

Aufgabe 12: Mit welchem anderen Baustein werden break und continue meistens kombiniert?

- ☐ a) Mit einer Verzweigung (if)
- ☐ b) Mit einer Funktion
- ☐ c) Mit einem Datentyp

Aufgabe 13: Gehe mit einer for-Schleife durch die Zahlen 1 bis 10 (range(1, 11)). Überspringe alle geraden Zahlen mit continue und gib nur die ungeraden aus. Höre mit break komplett auf, sobald du die 7 ausgegeben hast.

Dein Code:



Ein typischer Einsatz von break: **abbrechen, sobald ein Ziel erreicht ist**. Der Benutzer gibt eine Grenze vor, und die Schleife stoppt, sobald sie dort ankommt.

Aufgabe 14: Frage mit Zahl: eine ganze Zahl zwischen 0 und 10 ab. Gib die Zahlen von 0 aufwärts aus und brich mit break ab, sobald du die eingegebene Zahl ausgegeben hast (sie soll noch mit dabei sein).

Beispiel: Eingabe 4 → 0, 1, 2, 3, 4 (untereinander).

Dein Code:

Und ein typischer Einsatz von continue: **eine bestimmte Zahl auslassen**. Die Schleife läuft ganz normal durch, überspringt aber genau einen Wert.

Aufgabe 15: Frage mit Zahl: eine ganze Zahl zwischen 0 und 10 ab. Gib die Zahlen 0 bis 10 aus, aber überspringe mit continue die eingegebene Zahl.

Beispiel: Eingabe 4 → 0, 1, 2, 3, 5, 6, 7, 8, 9, 10 (die 4 fehlt).

Dein Code:



Verschachtelte Schleifen & Muster

Du kannst eine Schleife **in eine andere hineinsetzen** – das nennt man **verschachtelt**. Für **jeden** Durchlauf der äußeren Schleife läuft die **innere Schleife komplett** durch.

Wenn die äußere Schleife 3-mal läuft und die innere jeweils 3-mal, ergibt das $3 \cdot 3 = 9$ Durchläufe insgesamt.

Ein kleines Einmaleins mit zwei Schleifen – die innere Schleife (j) läuft für jedes i komplett durch:

Beispiel

```
for i in range(1, 4):  
    for j in range(1, 4):  
        print(i, "x", j, "=", i * j)
```

Aufgabe 16: Wie viele print-Zeilen erzeugt eine äußere Schleife mit 3 Durchläufen, in der eine innere Schleife mit 4 Durchläufen steckt?

- ☐ a) 7
- ☐ b) 12
- ☐ c) 3

Name: _____

Fach: IT



Für **Muster** ist ein Trick praktisch: Du kannst einen Text mit * **vervielfachen**. "*" * 4 ergibt "****".

So baust du mit **einer** Schleife eine wachsende Sternenreihe – die Zeilennummer bestimmt, wie viele Sterne ausgegeben werden.

Aufgabe 17: Gib mit einer for-Schleife folgendes Dreieck aus Sternen aus (5 Zeilen):

```
*  
**  
***  
****  
*****
```

Tipp: In Zeile *i* sollen *i* Sterne stehen – nutze "*" * *i*.

Dein Code:



Ein echtes **Raster** (Tabelle) entsteht erst mit einer Schleife **in** einer Schleife. Die äußere Schleife bestimmt die Reihe, die innere geht diese Reihe durch. So lässt sich zum Beispiel ein Einmaleins ausgeben.

Aufgabe 18: Gib mit zwei verschachtelten for-Schleifen das kleine Einmaleins für die Reihen 1 bis 3 aus. Jede Zeile soll die Form $i \times j = \text{Ergebnis}$ haben.

Erwartet (9 Zeilen):

```
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
2 x 1 = 2
...
3 x 3 = 9
```

Dein Code: